Chapter

Sorting

1. Bubble Sort

Intuition

Think of bubbles rising to the surface.

Compare adjacent elements:

```
5  3  8  4
^  ^
```

If they are in the wrong order, swap them. After one complete pass, the largest element reaches the end.

Example

```
[5, 3, 8, 4, 2]

Pass 1:
5 3 -> swap -> 3 5
5 8 -> no swap
8 4 -> swap -> 4 8
8 2 -> swap -> 2 8

[3,5,4,2,8]

8 is now fixed.

Repeat.

[3,5,4,2,8]
[3,4,2,5,8]
[3,2,4,5,8]
[2,3,4,5,8]
```

Code

```
void bubbleSort(vector<int>& a) {

    int n = a.size();

    for(int i = 0; i < n-1; i++) {

        bool swapped = false;

        for(int j = 0; j < n-i-1; j++) {

            if(a[j] > a[j+1]) {

                swap(a[j], a[j+1]);
                swapped = true;
            }
        }

        if(!swapped)
            break;
    }
}
```

Complexity

Case	Time
Best	O(n)
Average	$O(n^2)$

Worst	$O(n^2)$

Space: O(1) Stable: Yes

2. Selection Sort

Intuition

At every step, find the minimum element from the unsorted portion and put it at the beginning.

```
[5, 3, 8, 4, 2]
 ^
sorted portion
```

Find minimum = 2.

`Swap:`

```
[2,3,8,4,5]
```

Then find minimum from: [3,8,4,5] and continue.

Example

```
[5,3,8,4,2]

Minimum = 2
[2,3,8,4,5]

Minimum in remaining = 3
[2,3,8,4,5]

Minimum = 4
[2,3,4,8,5]

Minimum = 5
[2,3,4,5,8]
```

Code

```
void selectionSort(vector<int>& a) {

    int n = a.size();

    for(int i = 0; i < n-1; i++) {

        int minIndex = i;

        for(int j = i+1; j < n; j++) {

            if(a[j] < a[minIndex])
                minIndex = j;
        }

        swap(a[i], a[minIndex]);
    }
}
```

Complexity

Best: $O(n^2)$ Average: $O(n^2)$ Worst: $O(n^2)$

Space: O(1) Stable: No

3. Insertion Sort

Intuition

Think about arranging playing cards in your hand. You pick one card and insert it into its correct position among the cards already sorted.

```
[5 | 3 8 4 2]

5 is sorted.

Take 3:

[3 5 | 8 4 2]

Take 8:

[3 5 8 | 4 2]

Take 4:

[3 4 5 8 | 2]

Finally:

[2 3 4 5 8]
```

Code

```
void insertionSort(vector<int>& a) {

    int n = a.size();

    for(int i = 1; i < n; i++) {

        int key = a[i];

        int j = i-1;

        while(j >= 0 && a[j] > key) {

            a[j+1] = a[j];
            j--;
        }

        a[j+1] = key;
    }
}
```

Complexity

Case	Time
Best	O(n)
Average	O(n²)
Worst	O(n²)

Space: O(1) Stable: Yes

When useful?

Insertion sort is surprisingly useful when the array is: already almost sorted, small, or being sorted incrementally.

4. Merge Sort ■■■

This is very important.

Pattern: Divide and Conquer

Intuition

Instead of trying to sort the whole array at once:

```
[5,3,8,4,2,7,1,6]

divide it:
```

```
[5,3,8,4] [2,7,1,6]

Again:

[5,3] [8,4] [2,7] [1,6]

Again:

[5] [3] [8] [4] [2] [7] [1] [6]

A single element is already sorted.

Now merge sorted pieces.

[5] [3]
 |
[3,5]

Then:

[8] [4]
 |
[4,8]

Eventually:

[3,5,4,8]
 |
[3,4,5,8]
```

Code

```
void merge(vector<int>& a, int low, int mid, int high) {

    vector<int> temp;

    int i = low;
    int j = mid+1;

    while(i <= mid && j <= high) {

        if(a[i] <= a[j])
            temp.push_back(a[i++]);
        else
            temp.push_back(a[j++]);
    }

    while(i <= mid)
        temp.push_back(a[i++]);

    while(j <= high)
        temp.push_back(a[j++]);

    for(int k = low; k <= high; k++)
        a[k] = temp[k-low];
}


void mergeSort(vector<int>& a, int low, int high) {

    if(low >= high)
        return;

    int mid = low + (high-low)/2;

    mergeSort(a, low, mid);
    mergeSort(a, mid+1, high);

    merge(a, low, mid, high);
}
```

Call: `mergeSort(a, 0, a.size()-1);`

Example

```
[5,3,8,4]

Divide:
[5,3] [8,4]

Divide:
[5] [3] [8] [4]

Merge:
[3,5] [4,8]

Merge:
[3,4,5,8]
```

Complexity

Best: O(n log n) Average: O(n log n) Worst: O(n log n)

Space: O(n) Stable: Yes

Important DSA applications

Merge sort is not just for sorting. It is used for: Count Inversions, Reverse Pairs, Count Smaller Elements, Divide-and-conquer problems.

5. Quick Sort ■■■

Pattern: Divide and Conquer + Partition

Intuition

Choose a pivot. Put elements smaller than pivot to the left, elements greater than pivot to the right. Then recursively sort both sides.

```
Example:

[5,3,8,4,2,7,1,6]

Choose pivot = 5.

After partition:

[3,4,2,1] 5 [8,7,6]

Now recursively sort:

[3,4,2,1]

and:

[8,7,6]
```

Code — Lomuto Partition

```
int partition(vector<int>& a, int low, int high) {

    int pivot = a[high];

    int i = low-1;

    for(int j = low; j < high; j++) {

        if(a[j] < pivot) {

            i++;
```

```
            swap(a[i], a[j]);
        }
    }

    swap(a[i+1], a[high]);

    return i+1;
}


void quickSort(vector<int>& a, int low, int high) {

    if(low >= high)
        return;

    int p = partition(a, low, high);

    quickSort(a, low, p-1);
    quickSort(a, p+1, high);
}
```

Call: `quickSort(a, 0, a.size()-1);`

Example

```
[5,3,8,4,2]

Pivot = 2

[2,3,8,4,5]
 ^
pivot fixed

Then recursively sort remaining portion.
```

Complexity

Case	Time
Best	O(n log n)
Average	O(n log n)
Worst	O(n²)

Space: O(log n) average recursion stack. Stable: No

Why worst case?

If the pivot repeatedly becomes the smallest/largest:

```
[1,2,3,4,5]

you get:

1 | [2,3,4,5]
    2 | [3,4,5]
        ...

which becomes O(n^2).
```

6. Heap Sort ■■■

Pattern: Heap

Intuition

For ascending sorting, construct a Max Heap. Max heap property: parent >= children.

```
Example:

        9
```

```
      /   \
     7     8
    / \   /
   3   2 5
```

The largest element is always at the root.

Take the root and put it at the end.

Then rebuild the heap.

Important operation: Heapify

```
void heapify(vector<int>& a, int n, int i) {

    int largest = i;

    int left = 2*i + 1;
    int right = 2*i + 2;

    if(left < n && a[left] > a[largest])
        largest = left;

    if(right < n && a[right] > a[largest])
        largest = right;

    if(largest != i) {

        swap(a[i], a[largest]);

        heapify(a, n, largest);
    }
}
```

Heap Sort

```
void heapSort(vector<int>& a) {

    int n = a.size();

    // Build Max Heap
    for(int i = n/2 - 1; i >= 0; i--)
        heapify(a, n, i);

    // Extract maximum
    for(int i = n-1; i > 0; i--) {

        swap(a[0], a[i]);

        heapify(a, i, 0);
    }
}
```

Example

```
[4,10,3,5,1]

Build max heap:

        10
       /  \
      5    3
     / \
    4   1

Move 10 to end:

[1,5,3,4,10]

Heapify remaining:
```

```
[5,4,3,1,10]

Continue.

Final:

[1,3,4,5,10]
```

Complexity

Best: O(n log n) Average: O(n log n) Worst: O(n log n)

Space: O(1) auxiliary Stable: No

7. Counting Sort

When to use?

When values lie in a small range.

```
Example:

[4,2,2,8,3,3,1]

Maximum = 8.

Create frequency array:

value:  0 1 2 3 4 5 6 7 8
freq:   0 1 2 2 1 0 0 0 1

Then reconstruct.
```

Code

```
void countingSort(vector<int>& a) {

    int maxVal = *max_element(a.begin(), a.end());

    vector<int> freq(maxVal+1, 0);

    for(int x : a)
        freq[x]++;

    int index = 0;

    for(int x = 0; x <= maxVal; x++) {

        while(freq[x]--) {
            a[index++] = x;
        }
    }
}
```

Example

```
Input:
[4,2,2,8,3,3,1]

Frequency:
1 -> 1
2 -> 2
3 -> 2
4 -> 1
8 -> 1

Output:
[1,2,2,3,3,4,8]
```

Complexity

If range is k: Time: O(n + k) Space: O(k)

Important limitation

If: [1, 2, 999999999] counting sort becomes inefficient because the range is huge.

8. Radix Sort

Intuition

Instead of comparing complete numbers, sort numbers digit by digit. Usually start with units, then tens, then hundreds, and so on.

```
Example:

[170,45,75,90,802,24,2,66]

Sort by units:

[170,90,802,2,24,45,75,66]

Then tens.

Then hundreds.

Final:

[2,24,45,66,75,90,170,802]

Radix sort normally uses stable Counting Sort for each digit.
```

Code

```
void countingSort(vector<int>& a, int exp) {

    int n = a.size();

    vector<int> output(n);
    int count[10] = {};

    for(int x : a) {
        int digit = (x / exp) % 10;
        count[digit]++;
    }

    for(int i = 1; i < 10; i++)
        count[i] += count[i-1];

    for(int i = n-1; i >= 0; i--) {

        int digit = (a[i] / exp) % 10;

        output[count[digit]-1] = a[i];

        count[digit]--;
    }

    a = output;
}


void radixSort(vector<int>& a) {

    int maxVal = *max_element(a.begin(), a.end());

    for(int exp = 1;
        maxVal / exp > 0;
        exp *= 10) {
```

```
        countingSort(a, exp);
    }
}
```

Complexity

If n = number of elements, d = number of digits: Time: O(nd) Space: O(n + 10)

9. Bucket Sort

Intuition

Divide elements into several buckets based on their value.

```
Example:

[0.78,0.17,0.39,0.26,0.72,0.94,0.21,0.12,0.23,0.68]

Create buckets:

Bucket 0 -> 0.0 - 0.1
Bucket 1 -> 0.1 - 0.2
Bucket 2 -> 0.2 - 0.3
...

Put each element in its bucket.

Sort individual buckets.

Then concatenate them.
```

Code

For numbers in [0,1):

```
void bucketSort(vector<float>& a) {

    int n = a.size();

    vector<vector<float>> buckets(n);

    for(float x : a) {

        int index = x * n;

        buckets[index].push_back(x);
    }

    for(auto& bucket : buckets)
        sort(bucket.begin(), bucket.end());

    int index = 0;

    for(auto& bucket : buckets) {

        for(float x : bucket)
            a[index++] = x;
    }
}
```

Example

```
Input:
[0.42, 0.32, 0.23, 0.52, 0.25]

Buckets:

0.2 -> 0.23,0.25
0.3 -> 0.32
0.4 -> 0.42
```

```
0.5 -> 0.52

Output:
[0.23,0.25,0.32,0.42,0.52]
```

Complexity

Average: O(n + k) under suitable distribution. Worst: O(n²) depending on bucket distribution and internal sorting.

■ Most Important Comparison

Algorithm	Best	Average	Worst	Space	Stable
Bubble	O(n)	O(n²)	O(n²)	O(1)	Yes
Selection	O(n²)	O(n²)	O(n²)	O(1)	No
Insertion	O(n)	O(n²)	O(n²)	O(1)	Yes
Merge	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes
Quick	O(n log n)	O(n log n)	O(n²)	O(log n)*	No
Heap	O(n log n)	O(n log n)	O(n log n)	O(1)	No
Counting	O(n+k)	O(n+k)	O(n+k)	O(k)	Can be
Radix	O(nd)	O(nd)	O(nd)	O(n+k)	Yes
Bucket	O(n+k)**	O(n+k)**	O(n²)	O(n+k)	Depends

** Quick Sort recursion stack; worst case can become O(n). ** Under suitable distribution assumptions.*

■■ What You Actually Need for DSA

Don't spend equal time on all of them.

Must Know Completely

1. Merge Sort

Know:

```
Divide
|
Sort left
|
Sort right
|
Merge
```

And applications: Count Inversions, Reverse Pairs

2. Quick Sort

Know:

```
Pivot
|
Partition
|
Left + Pivot + Right
|
Recursion
```

Especially understand why worst case becomes O(n²).

3. Heap Sort

Know:

```
Build Heap
|
Maximum at root
```

```
|
Swap root with last
|
Heapify
|
Repeat
```

This also connects directly to Priority Queue / Heap problems.

Know the basics

Bubble → Selection → Insertion. You mainly need their intuition and complexity.

Know conceptually

Counting → Radix → Bucket. These are less frequently required in normal interview coding, but they are important for understanding non-comparison sorting.

■ One Very Important Concept

Sorting algorithms can be divided into:

Comparison-based

They decide order by comparing elements: a[i] < a[j]

Examples: Bubble, Selection, Insertion, Merge, Quick, Heap

Their general lower bound is: Ω(n log n) for comparison sorting.

Non-comparison-based

They exploit the values themselves: Counting, Radix, Bucket

They can achieve better than O(n log n) under suitable assumptions.

■■ Quick Recognition Trick

"Count inversions" → **Merge Sort**

"Reverse pairs" → **Modified Merge Sort**

"Need guaranteed O(n log n), O(1) extra space" → **Heap Sort**

"Partition around pivot" → **Quick Sort**

"Small range of integer values" → **Counting Sort**

"Sort digit by digit" → **Radix Sort**

"Values uniformly distributed" → **Bucket Sort**

"Nearly sorted array" → **Insertion Sort**